IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Tri-Level Hybrid DDQN-ALNS: A Hierarchical Learning-Augmented Metaheuristic for the Vehicle Routing Problem with Time Windows

HUYNH NHAT HUY[1], THI-LINH HO[2], NGUYEN NHAT HUY[3], AND NGUYEN THI BAO TRAN[4]

[1]Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam (e-mail: huynhnthuy@tdtu.edu.vn)
[2]Natural Language Processing and Knowledge Discovery Research Group, Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam (e-mail: hothilinh@tdtu.edu.vn)
[3]Mindx, Ho Chi Minh City, Vietnam (e-mail: huynguyenwork14@gmail.com)
[4]Vinsmart future, Ho Chi Minh City, Vietnam (e-mail: tranntb.se@gmail.com)

Corresponding author: Thi-Linh Ho (e-mail: hothilinh@tdtu.edu.vn).

This research is supported by Ton Duc Thang University.

ABSTRACT The Vehicle Routing Problem with Time Windows (VRPTW) is commonly solved under a lexicographic objective: minimize the number of vehicles (N_{V}) first, then total travel distance (D_{T}). We ask whether learned control inside Adaptive Large Neighborhood Search (ALNS) pays for its compute when the controllers start from random weights in every run. The test case, Tri-Level Hybrid DDQN-ALNS, adds three online-learning controllers to ALNS: a macro plateau controller, a micro operator controller built on a Dueling Double Deep Q-Network , and a Learned Acceptance Criterion that can accept moves Simulated Annealing rejects. The search also uses ejection chains, set partitioning with HiGHS, a frozen Graph Neural Network edge heatmap, and a classical Guided Ejection Search phase. That phase was added after earlier benchmark results were seen and kept by a pre-specified rule, so its inclusion was not independently tested. We claim neither a new search principle nor a convergence guarantee. We evaluate the method under equal wall-clock budgets of 10, 60 and 300 s on 74 Solomon and Gehring-Homberger instances with five seeds per instance, against ALNS-Base, PyVRP-ILS and NeuroLKH-adapted, counting only incumbents certified in the original problem space before the cutoff. At 300 s, the proposed configuration did not differ detectably on fleet size (fleet score, 98.33% interval; positive favors it) from ALNS-Base (-0.027 $[-0.065, 0.011]$), PyVRP-ILS ($+0.008$ $[-0.098, 0.117]$) or NeuroLKH-adapted (-0.068 $[-0.142, 0.033]$). In a factorial ablation on 18 instances, no primary factorial contrast's fleet-score interval excluded zero at 300 s.

INDEX TERMS Adaptive Large Neighborhood Search, Deep Reinforcement Learning, Double DQN, Semi-Markov Decision Process, Metaheuristics, Vehicle Routing Problem with Time Windows.

I. INTRODUCTION

THE Vehicle Routing Problem with Time Windows (VRPTW) is an NP-hard combinatorial optimization problem that arises in freight logistics and last-mile parcel delivery [1], [2]. Each route must respect vehicle capacity and the time window of every customer it visits.

Commercial freight planning often treats the objective as lexicographic: dispatchers prioritize minimizing the total number of deployed vehicles (N_{V}) before minimizing total travel distance (D_{T}) [3], [4]. Formally, letting $\mathcal{S}$ denote the space of feasible routing solutions, a solution $s_1 \in \mathcal{S}$ is strictly preferred over $s_2 \in \mathcal{S}$ (denoted $s_1 \prec s_2$) if and only if:

$$N_{\mathrm{V}}(s_1) < N_{\mathrm{V}}(s_2) \ \vee \ \big(N_{\mathrm{V}}(s_1) = N_{\mathrm{V}}(s_2) \wedge D_{\mathrm{T}}(s_1) < D_{\mathrm{T}}(s_2)\big) \tag{1}$$

This ordering reflects the cost structure of commercial transport, where vehicle acquisition, maintenance, insurance and driver wages outweigh marginal fuel cost.

Adaptive Large Neighborhood Search (ALNS) [5], [6] is a leading metaheuristic for VRPTW. It repeatedly destroys and repairs parts of a solution, and its standard form keeps solutions feasible. Standard implementations choose operators with roulette-wheel or bandit rules that aggregate reward scores over coarse segments. These rules do not condition on the current search state, such as route frag-

mentation, temporal slack or capacity use, and they do not coordinate the several steps needed to leave a plateau on which removing a vehicle means passing through worse intermediate solutions. Learned controllers that condition these choices on the search state have been proposed [16], [17], [30]. They are not free: every network update and every forward pass takes time from the search itself, so under a fixed wall-clock budget a learned controller must gain more than it costs.

Neural Combinatorial Optimization (NCO) takes a different route and learns to construct routes end to end, with Pointer Networks [7], Attention Models [8] and constructive policies [9]. Such models are usually trained to minimize distance, and hard time windows complicate autoregressive decoding [10], [11]. We do not evaluate end-to-end solvers here; we study the other option, learning inside a search that already handles the constraints.

This paper asks whether learned control inside ALNS pays for its compute when the controllers start from random weights in every run. The test case is Tri-Level Hybrid DDQN-ALNS, which places state-conditioned reinforcement learning inside the search loop of ALNS , keeps the ALNS neighborhood operators and returns only solutions validated as feasible. The search has three tiers: a learned macro controller for plateaus, a learned micro controller for operator selection, and a heuristic search tier. At 300 s our answer is negative: the method did not differ detectably on fleet size from ALNS-Base, PyVRP-ILS or NeuroLKH-adapted, and the factorial ablation does not establish a benefit of the learned components at that budget. The ablation is also less informative than it looks: in a separate measurement at 300 s the acceptance model never changed an acceptance decision and the macro controller made no decision on the R2 and RC2 instances, so parts of the learned layer were hardly exercised. The one signal for learning is at 60 s: in a separate test on new time-window variants, Micro-DDQN used fewer vehicles than a rule-based policy in the bare configuration, and the test did not detect a difference in the full context (Section V-E).

The primary contributions of this paper are summarized as follows:

1) We specify a control layer for ALNS in which a macro plateau controller, a micro operator controller and an acceptance model learn online from random weights in every run, with no offline policy training. Destroy-repair search, Simulated Annealing, ejection chains and set partitioning are established methods, and we do not claim a new search principle. What we study is whether such cold-start control pays for its compute within a time budget.
2) We evaluate the method under equal wall-clock budgets of 10, 60 and 300 s on 74 benchmark instances (56 Solomon-100, 12 Homberger-200, and 6 Homberger-400) with five seeds, against ALNS-Base, PyVRP-ILS and NeuroLKH-adapted. All cases of an instance, seed and cutoff run as one block, and only incumbents certified in the original problem space before the cutoff count. At 300 s, the method did not differ detectably on fleet size from any of the three comparators, and we report this as a negative result.
3) We separate learning from heuristic machinery with a wall-clock factorial ablation over six components at 60 and 300 s, including a 2×2 contrast of the learning and heuristic bundles. The Guided Ejection Search phase is on in every case, so the ablation cannot isolate its effect. No primary contrast excluded zero at 300 s. In a separate measurement on the ablation instances, the LAC classifier never changed an acceptance decision at that budget, so the LAC contrast is uninformative rather than evidence that LAC has no effect.
4) We report the limits of the method and of the evidence: where it does not help, how results depend on the budget, the method's earlier dependence on reference fleet sizes, the machine-speed variation we measured, and the way test results influenced development.

The remainder of this paper is organized as follows: Section II reviews related literature. Section III presents the problem formulation. Section IV describes the method. Section V reports the benchmark results, statistical analysis and ablation. Section VI discusses where the method does and does not help, its practical meaning, and its limitations. Finally, Section VII concludes the paper.

II. RELATED WORK

A. CLASSICAL METAHEURISTICS AND ALNS FOR VRPTW

Metaheuristics have long dominated practical VRPTW optimization [4]. Building on Solomon's insertion heuristics [2] and Shaw's Large Neighborhood Search [18], Ropke and Pisinger [5], [6], [19] formalized Adaptive Large Neighborhood Search (ALNS), in which removal and insertion operators compete for selection via roulette-wheel probabilities updated from historical performance. For fleet minimization, Guided Ejection Search [20] and penalty-based edge assembly memetic algorithms [21] target fleet size directly, while matheuristic hybrids periodically solve Set Partitioning models over pools of elite routes [22]–[24]. More recently, Slack Induction by String Removals [12] demonstrated that specialized ruin operators can rival more complex methods.

In standard ALNS, operator weights are exponentially smoothed scores updated over coarse iteration segments, so credit arrives late and in aggregate. The selection policy is *stateless* : it ignores route topology, cluster fragmentation and time-window tightness. Turkeš et al. [25] found that this adaptive mechanism yields only marginal gains over non-adaptive LNS . That result leaves open whether a better selection policy would help or whether operator selection matters little, which is one reason we measure it directly.

B. NEURAL COMBINATORIAL OPTIMIZATION AND END-TO-END DEEP RL

Neural Combinatorial Optimization (NCO) learns routing policies using Pointer Networks [7], [26], reinforcement

TABLE 1. Taxonomy of Representative Methodologies for VRPTW and Related Routing Problems

Methodology Category	Representative Works	Hard TW Feasible	Objective Paradigm	State-Aware Selection	Hierarchical Control	Learned Accept	Per-Step Compute Cost
Classical Metaheuristics	ALNS [5], [6]	✓	Lexicographic	× (Stateless)	×	× (SA)	Minimal (Heuristic)
	SISR [12], HGS [13]	✓	Lexicographic	× (Rule-based)	×	× (SA/Population)	Minimal (Heuristic)
End-to-End Neural Solvers	AM [8], DRL-VRP [9]	× (Penalized)	Scalarized	N/A	×	N/A	Very Low (One-shot)
	Neural VRPTW [10], [11]	× (Mask-fragile)	Scalarized	N/A	×	N/A	Very Low (One-shot)
Single-Policy Hybrid RL-LNS	L2I [14]	N/A (No TW)	Distance	✓	× (Single-agent)	× (None)	Moderate (Policy NN)
	Neural LNS [15]	N/A (No TW)	Distance	✓	× (Single-agent)	× (SA/Greedy)	Moderate (Policy NN)
	Graph RL ALNS [16], DR-ALNS [17]	Problem-dependent	Problem-dependent	✓	× (Single policy)	Partly (DR-ALNS)	Moderate (Policy NN)
Proposed Tri-Level Hybrid DDQN-ALNS	**This Work**	**Validated (Returned plan)**	**Lexicographic**	✓	✓ **(Two controllers)**	✓ **(LAC)**	**High (RL/MIP)**

learning on dynamic customer states [9], Attention Models [8], and Graph Neural Networks [27], [28]. Extensions of NCO to VRPTW [10], [11] give fast inference on small instances ; hard time windows complicate their decoding, and they are usually trained on scalarized costs instead of fleet size first ($N_V \succ D_T$) . We do not evaluate these solvers.

C. LEARNING-AUGMENTED METAHEURISTICS

The "Learning to Optimize" line of work [29] embeds neural models inside metaheuristic search loops for operator selection [14], [30], solution perturbation [31], and learned destroy policies [15]. Closest to this work, Johnn et al. [16] formulate operator choice in ALNS as a Markov decision process solved with deep reinforcement learning and graph neural networks, and DR-ALNS [17] learns a policy that selects operators, adjusts parameters and controls the acceptance criterion, so learned influence on acceptance is not new. Our design differs in three respects: separate controllers for plateau regimes and operator pairs in place of one policy, an acceptance model added to Simulated Annealing by a disjunctive rule, and online learning from random weights in every run with no prior policy training for these controllers. We did not run these methods, and these differences are not evidence that our method performs better.

D. POSITIONING OF THIS WORK

Table 1 places Tri-Level Hybrid DDQN-ALNS in the routing literature. Our approach combines validated feasibility of returned solutions with two controllers at different timescales and a Learned Acceptance Criterion. Separating macro and micro control addresses credit assignment across timescales [32]: the reward for leaving a plateau of several hundred iterations comes long after any single destroy-repair decision. That motivates the separation, although we do not compare against a single flat policy. The macro tier's semi-MDP formulation over $\Delta_{\text{seg}} = 100$-iteration options separates plateau control from operator selection. Table 1 also shows the cost: online network updates and MIP solves consume part of each time budget. Section V measures whether this pays off.

III. PROBLEM FORMULATION

The Vehicle Routing Problem with Time Windows (VRPTW) is formally defined on a complete directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$. The vertex set $\mathcal{V} = \mathcal{N} \cup \{0, n+1\}$ comprises customer nodes $\mathcal{N} = \{1, 2, \ldots, n\}$, origin depot node 0, and destination depot node $n+1$. The physical depot is modeled as two separate nodes (0 and $n+1$) to represent route departure and return without self-loops. The arc set is $\mathcal{A} = \{(i,j) : i,j \in \mathcal{V}, i \neq j, i \neq n+1, j \neq 0\}$.

Each customer $i \in \mathcal{N}$ has a positive demand $q_i > 0$, service duration $s_i \geq 0$, and service time window $[e_i, l_i]$, where e_i and l_i denote the earliest and latest allowable service start times, respectively. Depot nodes satisfy $q_0 = q_{n+1} = 0$, $s_0 = s_{n+1} = 0$, and $[e_0, l_{n+1}]$ defines the global planning horizon. Each arc $(i,j) \in \mathcal{A}$ has travel distance $c_{ij} \geq 0$ and travel time $t_{ij} \geq 0$. A homogeneous fleet of K vehicles, denoted by $\mathcal{K} = \{1, \ldots, K\}$, is stationed at the depot, with each vehicle possessing carrying capacity Q.

A. MATHEMATICAL MODEL AND DECISION VARIABLES

The optimization model employs binary routing variables $x_{ij}^k \in \{0,1\}$ (1 if vehicle $k \in \mathcal{K}$ traverses arc $(i,j) \in \mathcal{A}$, and 0 otherwise) and continuous schedule variables $w_i^k \geq 0$ specifying the time at which vehicle $k \in \mathcal{K}$ begins service at node $i \in \mathcal{V}$. The primary objective is lexicographic: first minimizing the number of deployed vehicles (N_V), and secondarily minimizing total travel distance (D_T):

$$\text{lex min} \left(\sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{N}} x_{0j}^k, \quad \sum_{k \in \mathcal{K}} \sum_{(i,j) \in \mathcal{A}} c_{ij} x_{ij}^k \right) \tag{2}$$

The solver evaluates this hierarchy through a scalarized surrogate with a vehicle penalty $\mathcal{M}$. The surrogate ranks two solutions as the lexicographic order does whenever $\mathcal{M}$

exceeds the difference between their total distances. We use $\mathcal{M} = 10^5$, which is above the total distance of every solution evaluated in this study; this concerns the comparison of evaluated solutions and implies no optimality guarantee:

$$\min \sum_{k\in\mathcal{K}} \left(\mathcal{M} \sum_{j\in\mathcal{N}} x_{0j}^k + \sum_{(i,j)\in\mathcal{A}} c_{ij} x_{ij}^k \right) \tag{3}$$

subject to:

$$\sum_{k\in\mathcal{K}} \sum_{j\in\mathcal{V}, j\neq i} x_{ij}^k = 1, \quad \forall i \in \mathcal{N} \tag{4}$$

$$\sum_{j\in\mathcal{N}\cup\{n+1\}} x_{0j}^k \leq 1, \quad \forall k \in \mathcal{K} \tag{5}$$

$$\sum_{j\in\mathcal{N}\cup\{0\}} x_{j,n+1}^k = \sum_{j\in\mathcal{N}\cup\{n+1\}} x_{0j}^k, \quad \forall k \in \mathcal{K} \tag{6}$$

$$\sum_{i\in\mathcal{V}, i\neq h} x_{ih}^k - \sum_{j\in\mathcal{V}, j\neq h} x_{hj}^k = 0, \quad \forall h \in \mathcal{N},\ k \in \mathcal{K} \tag{7}$$

$$\sum_{i\in\mathcal{N}} q_i \sum_{j\in\mathcal{V}, j\neq i} x_{ij}^k \leq Q, \quad \forall k \in \mathcal{K} \tag{8}$$

$$w_i^k + s_i + t_{ij} - M_{ij}(1 - x_{ij}^k) \leq w_j^k, \quad \forall (i,j) \in \mathcal{A},\ k \in \mathcal{K} \tag{9}$$

$$e_i \leq w_i^k \leq l_i, \quad w_i^k \geq 0, \quad \forall i \in \mathcal{V},\ k \in \mathcal{K} \tag{10}$$

$$x_{ij}^k \in \{0,1\}, \quad \forall (i,j) \in \mathcal{A},\ k \in \mathcal{K} \tag{11}$$

Constraint (4) ensures that every customer is visited exactly once. Constraints (5)–(6) enforce that each vehicle departs from origin depot 0 and arrives at destination depot $n+1$ at most once. Flow conservation across all customer nodes is guaranteed by (7). Constraint (8) prevents vehicle capacity violations. Schedule propagation is maintained via (9), where $M_{ij} = \max\{0, l_i + s_i + t_{ij} - e_j\}$ defines the tightest valid Big-M upper bound. Customer time-window compliance and domains are governed by (10)–(11); if a vehicle arrives at customer i prior to e_i, it incurs an idle waiting time such that service begins at $w_i^k = e_i$. Although w_i^k is indexed per vehicle to match x_{ij}^k, constraint (4) ensures that exactly one vehicle serves each customer i, so only one w_i^k is active in any feasible plan.

IV. PROPOSED METHODOLOGY

This section describes Tri-Level Hybrid DDQN-ALNS , which combines online reinforcement learning controllers with classical search for the lexicographic VRPTW objective . Derivations are in Supplementary Sections 1–4, and the full algorithm is in Supplementary Section 5.

A. TRI-LEVEL ARCHITECTURE OVERVIEW

The method has three tiers, shown in Fig. 1 :

1) Macro Decision Tier (Plateau Controller π_{macro}): formulated as an aggregated semi-MDP over segments of length $\Delta_{\text{seg}} = 100$, where a Dueling Double DQN [33], [34] with Prioritized Experience Replay [35] observes search progress and switches among 7 search modes;
2) Micro Selection Tier (Operator Controller π_{micro} & LAC): operates per iteration t, using a Micro Dueling DDQN with a normalized Softmax-Entropy gate to select a destroy–repair operator pair from a 20-dimensional state , together with a Learned Acceptance Criterion (LAC) trained on delayed outcomes; and
3) Heuristic Search Tier (Granular ALNS & Route-Pool Recombination): executes ruin-and-recreate operations with $O(1)$-per-query Savelsbergh forward-slack insertion checks on cached route profiles, restricts moves to the $k = 15$ nearest customers, and periodically solves a time-limited Set Partitioning formulation over elite routes via HiGHS .

B. MACRO LEVEL: PLATEAU CONTROLLER (AGGREGATED SMDP FORMULATION)

We formulate the macro tier as a Semi-Markov Decision Process [32] over an Aggregated State Representation (Aggregated SMDP) defined by the tuple $(\mathcal{S}_{\text{macro}}, \mathcal{A}_{\text{macro}}, P_{\text{macro}}, R_{\text{macro}}, \gamma_{\text{macro}})$. Because the underlying combinatorial route pool and fine-grained trajectory history are projected into a 12-dimensional feature space $\mathbf{s}_k^{\text{macro}}$, the decision tier operates over a compact aggregated state representation with temporal options of fixed duration $\Delta_{\text{seg}} = 100$ low-level ALNS iterations. The controller acts only once the search has stagnated for a minimum number of iterations; before that, segments run in the default mode and are not used as macro transitions. The temporally extended transition kernel $P_{\text{macro}}(\mathbf{s}_{k+1}^{\text{macro}} \mid \mathbf{s}_k^{\text{macro}}, a_k)$ denotes the distribution of the next observation after the 100 underlying stochastic destroy–repair steps . The learner discounts per macro decision with $\gamma_{\text{macro}} = 0.95$, used both in the DDQN target and in the shaping term below, rather than with $\gamma^{\Delta_{\text{seg}}}$.

The aggregated formulation is a modeling device, not a verified Markov model. The 12 features are a lossy summary of the full search state, which also includes the current routes, the route pool, the operator weights, the network parameters and the random-number state. Distinct search states therefore share an observation, and the next observation depends on components that the controller does not see, so $\mathbf{s}_k^{\text{macro}}$ is not in general a sufficient statistic and the macro process need not satisfy the Markov property. The controller is better described as a learned policy over a partially observed, state-aggregated process [36]. We provide no convergence guarantee and no bound on the approximation error. Such guarantees are not available in general for Q-learning over aggregated or partial observations, and the online DDQN additionally uses nonlinear function approximation, experience replay and target networks. The value of the controller is therefore assessed empirically through the ablation study (Section V-E).

1) Macro State Representation

At the start of segment k, the macro controller observes a 12-dimensional state vector $\mathbf{s}_k^{\text{macro}} \in \mathbb{R}^{12}$, covering search progress, fleet size and route structure:

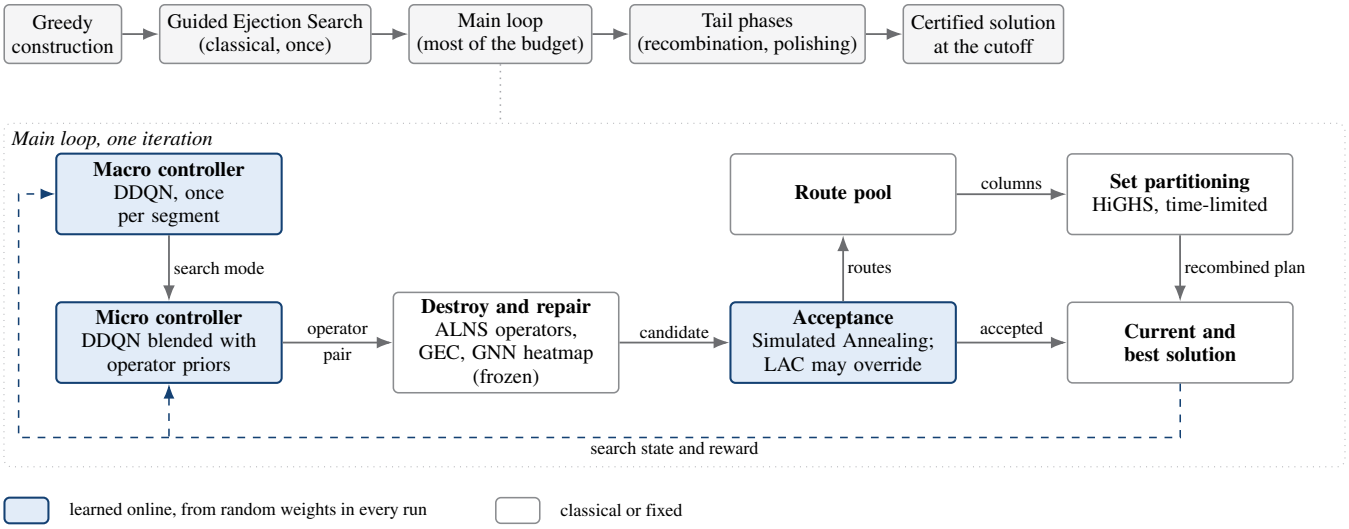


FIGURE 1. Tri-Level Hybrid DDQN-ALNS . Top: the phases of one run within the wall-clock budget. Bottom: one iteration of the main loop. Shaded components learn online from random weights in every run; the others are classical or fixed, and the GNN heatmap comes from a frozen pretrained model. The macro controller chooses a search mode once per segment of iterations, and one mode triggers set partitioning over the route pool. Dashed arrows carry the search state and reward to the controllers.

$$\mathbf{s}_k^{\text{macro}} = \Big[\min\Big(\frac{\text{no_imp}}{\text{patience}}, 1\Big),\ \min\Big(\frac{C_{\text{cur}} - C_{\text{best}}}{C_{\text{best}}}, 1\Big),\ \min\Big(\frac{T}{T_0}, 1.5\Big),\ \text{imp_rate},\ \min\Big(\frac{N_{\text{V,cur}}}{N_{\text{V,init}}}, 2\Big),\ rb,\ lb,\ \tau_{\text{tight}},\ \bar{s},\ \rho_{\text{fill}},\ \rho_{\text{pool}},\ \frac{k}{K_{\max}}\Big] \tag{12}$$

where no_imp is the number of consecutive unimproved iterations, T/T_0 is the normalized annealing temperature, $rb \in [0,1]$ and $lb \in [0,1]$ measure spatial route separation and capacity balance, $\tau_{\text{tight}} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}(l_i - e_i < 0.5 \cdot d_{\max})$ is the fraction of customers whose time-window width is below half the largest distance $d_{\max}$ of the instance, $\bar{s}$ is the average forward time slack, ρ_{fill} is the average vehicle capacity utilization, and $\rho_{\text{pool}} = \min(|\mathcal{P}|/\mathcal{P}_{\max}, 1.0)$ tracks the route pool fill ratio.

2) Macro Action Space & Search Regimes

The macro action space $\mathcal{A}_{\text{macro}}$ comprises 7 search modes. Each mode scales the destroy size, multiplies the temperature once per segment, biases the operator priors and sets the local-search passes. With (destroy scale, temperature factor) in parentheses, the modes are: a_0 (DEFAULT , 1.00, 1.00); a_1 (INTENSIFY, 0.70, 0.98); a_2 (DIVERSIFY , 1.35, 1.08), which also permits a one-vehicle increase in the current solution; a_3 (TW_RESCUE , 1.10, 1.05), with priors favoring time-window urgency removal and time-window-aware repair; a_4 (POOL_RECOMBINE , 0.90, 1.01), which may trigger Set Partitioning; a_5 (ROUTE_REDUCE, 0.95, 1.02), with priors favoring route-elimination operators, a Generalized Ejection Chain attempt at every iteration, penalized acceptance, and possible recombination; and a_6 (INFEASIBLE_DESCENT , 1.10, 1.05), which is available only during the first search phase described below and maps to DEFAULT afterwards. The segment length is 100 iterations, or 130 on instances with wide time windows.

3) Search Phases

The search changes behavior over the budget. In the first phase, candidates are accepted by a penalized Simulated Annealing rule on $C_{\text{pen}}(s) = C(s) + \lambda\,(v_{\text{cap}}(s) + v_{\text{tw}}(s))$, so the current solution may be infeasible. The multiplier λ is updated every 100 iterations, by a factor of 0.90 when more than half of recent candidates are feasible and 1.15 otherwise, and is bounded by $[5, 200]$ for instances with tight time windows and $[0.5, 50]$ for others. The phase ends after 60% of a nominal 10,000 iterations, when the fleet reaches its floor, or, with no known floor, after 250 iterations without a new best solution (and at least 200 iterations in total). LAC is not consulted in this phase, and mode a_5 also keeps penalized acceptance afterwards.

The main destroy-repair loop receives 80% of the time budget. The remaining 20% goes to tail phases: pool recombination, polishing at fixed fleet size, dense column generation for the pool, route-elimination heuristics, and distance polishing. ALNS-Base has no tail phases. A few settings also adapt to the instance: the initial temperature is scaled by 0.8 for tight and 1.2 for wide time windows, and the plateau trigger is 1.5 times longer for $N \geq 200$.

4) Multi-Objective Segment Reward & Surrogate Policy Optimization

The segment reward R_{seg} weighs vehicle reduction against distance savings through an adaptive fleet pressure factor $\lambda_{\text{fleet}} \in [0,1]$:

$$R_{\text{seg}} = 0.30\big[\text{Base} + \lambda_{\text{fleet}}\Psi_{\text{NV}} + (1-\lambda_{\text{fleet}})\Psi_{\text{TD}} + \text{Bonus}_{\text{NV}}\big] + \Delta\Phi \tag{13}$$

where $\Psi_{\mathrm{NV}} = \sum_{j=1}^{4} \beta_{\mathrm{NV},j} \Delta X_j$ and $\Psi_{\mathrm{TD}} = \sum_{j=1}^{4} \beta_{\mathrm{TD},j} \Delta X_j$ linearly weight improvements across best-so-far and incumbent fleet size (ΔN_{V}) and travel distance (ΔC), with a representative primary fleet coefficient $\beta_{\mathrm{NV},1} = 8.0$. The baseline term Base penalizes intensive local search passes and unproductive stagnation, while $\mathrm{Bonus}_{\mathrm{NV}}$ provides a discrete reward of $+15$ per vehicle removed from the best solution and $+5$ per vehicle removed from the current solution. All coefficients are listed in Supplementary Section 6.

Equation (13) has two fleet rewards. The dense term ($\lambda_{\mathrm{fleet}} \Psi_{\mathrm{NV}}$) rewards moves that balance route loads and narrow the fleet gap to the incumbent . The sparse term ($\mathrm{Bonus}_{\mathrm{NV}}$) pays only when a vehicle is removed, so that signal is not diluted by routine changes in distance.

a: Reward Constants

The reward constants were set by hand during development, and Section V-A states what we can and cannot document about them and how the test set influenced later development.

Evaluation and training use different objectives. Incumbent replacement, elite archive admission and final selection use the lexicographic order $f(s) = (N_{\mathrm{V}}(s), D_{\mathrm{T}}(s))$, so fleet size always comes before distance. The policy instead receives the scalar reward R_{seg}, modulated by λ_{fleet}. When the current fleet is above the best ($N_{\mathrm{V,cur}} > N_{\mathrm{V,best}}$), $\lambda_{\mathrm{fleet}} \to 1$ concentrates the reward on removing vehicles; once they match ($N_{\mathrm{V,cur}} = N_{\mathrm{V,best}}$), $\lambda_{\mathrm{fleet}} \leq 0.5$ moves the weight to distance.

We also add potential-based reward shaping $\Delta\Phi = \gamma_{\mathrm{macro}} \Phi(\mathrm{s}_{k+1}^{\mathrm{macro}}) - \Phi(\mathrm{s}_{k}^{\mathrm{macro}})$ to speed up learning without creating artificial reward cycles in the surrogate MDP . This shaping applies to the *macro surrogate objective* [37] . It does not give policy invariance with respect to the lexicographic VRPTW objective , which the evaluator enforces separately. The fleet pressure factor rises when the current fleet is above the best:

$$\lambda_{\mathrm{fleet}}(s) = \sigma\left(8.0 \cdot \frac{N_{\mathrm{V,cur}} - N_{\mathrm{V,best}}}{N_{\mathrm{V,init}}}\right) \tag{14}$$

where $\sigma(\cdot)$ is the logistic sigmoid, with the state potential defined as:

$$\begin{aligned}\Phi(s) =& -15\, \lambda_{\mathrm{fleet}}(s) \frac{\max(N_{\mathrm{V}} - N_{\mathrm{V,best}}, 0)}{N_{\mathrm{V,init}}} \\ & -0.18\big(1 - \lambda_{\mathrm{fleet}}(s)\big) \\ & \times \mathrm{clip}\left(\frac{D_{\mathrm{T}} - D_{\mathrm{T,best}}}{D_{\mathrm{T,best}}} \cdot 100, -25, 25\right)\end{aligned} \tag{15}$$

The Macro Plateau Controller trains its own Dueling Double DQN (Q_{macro}) with prioritized replay buffer $\mathcal{D}_{\mathrm{macro}}$, taking Adam steps with a cosine-annealed learning rate at the end of each macro segment.

C. MICRO LEVEL: OPERATOR CONTROLLER & ACTION BLENDING

1) Micro State Representation

At each iteration t, the operator controller observes a 20-dimensional state vector $\mathbf{s}_t^{\mathrm{micro}} \in \mathbb{R}^{20}$ (Table 2), which describes search progress and route structure.

2) Composite Action Space

The micro action space $\mathcal{A}_{\mathrm{micro}}$ comprises $N_D \times N_R = 13 \times 5 = 65$ composite heuristic pairs, combining 13 Destroy Operators (Random, Worst-cost, Shaw spatiotemporal relatedness, Route portion removal, Time-window urgency removal, Route-Depletion Destroy, Route dispersion elimination, Costly route elimination, Cross-route Shaw, Route merge sampling, Route absorb disruption, Neural worst, Neural Shaw) with 5 Repair Operators (Greedy insertion, 2-Regret, 3-Regret, Time-window greedy, Forward-time-slack greedy).

3) Dueling DDQN Architecture & Policy-Concentration Blending

The Micro Q-network adopts a Dueling architecture [38] with layer normalization:

$$Q(s,a;\theta) = V(s;\theta_v) + \left(A(s,a;\theta_a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a';\theta_a)\right) \tag{16}$$

A normalized Softmax-Entropy Policy-Concentration Gate balances the Q-values against the exploration priors. Let $\pi_\theta(a \mid s) = \mathrm{softmax}(Q(s,a)/\tau_{\mathrm{soft}})$ be the Boltzmann policy at softmax temperature $\tau_{\mathrm{soft}} = 1.0$. The normalized Shannon entropy is defined as:

$$\mathcal{H}(\pi_\theta(\cdot \mid s)) = -\frac{1}{\ln|\mathcal{A}|}\sum_{a\in\mathcal{A}} \pi_\theta(a \mid s) \ln \pi_\theta(a \mid s) \in [0,1] \tag{17}$$

We define the policy-concentration weight as $w_{\mathrm{conf}}(s) = 1.0 - \mathcal{H}(\pi_\theta(\cdot \mid s))$. High concentration ($w_{\mathrm{conf}} \to 1$) indicates that the Boltzmann policy is sharply peaked around a dominant operator pair, whereas low concentration ($w_{\mathrm{conf}} \to 0$) signals an exploratory regime.

The action-selection score blends the Q-values with operator priors and the mean of a Thompson-sampling

TABLE 2. 20-Dimensional Micro State Vector $\mathbf{s}_t^{\mathrm{micro}}$

Idx	Sym	Definition	Idx	Sym	Definition
s_1	ΔC_{rel}	$(C_{\mathrm{cur}} - C_{\mathrm{best}})/C_{\mathrm{best}}$	s_{11}	ρ_{fill}	Capacity fill ratio
s_2	$\mathrm{Ratio}_{\mathrm{NV}}$	$N_{\mathrm{V,cur}}/N_{\mathrm{V,init}}$	s_{12}	ρ_{pool}	Pool fill ratio $\lvert\mathcal{P}\rvert/\mathcal{P}_{\max}$
s_3	$\mathrm{Prog}_{\mathrm{global}}$	$\min(t/10000, 1)$	s_{13}	m_{idx}	Active macro mode index
s_4	$\mathrm{Prog}_{\mathrm{seg}}$	$(t \bmod \Delta_{\mathrm{seg}})/\Delta_{\mathrm{seg}}$	s_{14}	$\Delta N_{\mathrm{V,rel}}$	$(N_{\mathrm{V,cur}} - N_{\mathrm{V,best}})/N_{\mathrm{V,init}}$
s_5	T_{norm}	T/T_0	s_{15}	$\mathrm{Imp}_{\mathrm{rec}}$	Recent improvement rate
s_6	$\mathrm{Stag}_{\mathrm{ratio}}$	no_imp/patience	s_{16}	$\mathrm{Gini}_{\mathrm{len}}$	Gini of route loads
s_7	rb	Route separation index	s_{17}	μ_{slack}	Mean customer forward slack
s_8	lb	Capacity load balance	s_{18}	σ_{slack}	Std of forward slacks
s_9	τ_{tight}	Frac tight TW customers	s_{19}	$\mathrm{Frac}_{\mathrm{cap}}$	Fraction routes $> 90\%$ cap
s_{10}	$\bar{s}$	Normalized mean slack	s_{20}	$\mathrm{Var}_{\mathrm{dist}}$	Normalized variance of route distances

posterior through this gate, then adds a variance-aware bonus adapted from UCB [39] and UCB-V [40] :

$$\begin{aligned} Q_{\text{blend}}(s,a) &= w_{\text{conf}}(s)\, Q(s,a) + (1 - w_{\text{conf}}(s))\big[\alpha_{\text{p}} \ln P(a) \\ &\quad + \beta_{\text{b}}\, \bar{\mu}_{\text{bandit}}(a)\big], \\ Q_{\text{eff}}(s,a) &= Q_{\text{blend}}(s,a) + \alpha_{\text{u}}\, \frac{U(a) - \bar{U}}{\sigma_U}, \\ U(a) &= \hat{\mu}(a) + c_{\text{ucb}}\, \sigma_a \sqrt{\ln N_t / N_a(t)}, \end{aligned} \tag{18}$$

where $\alpha_{\text{p}} = 0.55$ scales prior log-probabilities, $\beta_{\text{b}} = 0.20$ weights the bandit posterior mean, $\alpha_{\text{u}} = 0.35$ weights the standardized bonus over the actions, $c_{\text{ucb}} = 1.0$, $\hat{\mu}(a)$ is a running mean reward, and $\sigma_a = \sqrt{M_{2,a}/\max(N_a(t) - 1, 1)}$ is an online standard deviation tracked with Welford's algorithm. Action counters start at 0.5 and decay by 0.993 per step, and $N_t = \sum_a N_a(t)$.

Two properties of this rule follow directly. The Q-values and log-priors are not standardized before the gate, so the balance between them depends on the scale of the learned values. The bonus is not gated, so it continues to affect selection when the policy is concentrated.

The first 256 transitions select pairs from the bandit with the prior. After that, the controller explores with probability ε, sampling from the prior and bandit, with ε decaying from 0.35 to 0.02 by a factor of 0.9996 per step. It otherwise takes the pair that maximizes Q_{eff}. The Micro reward has the same form as R_{seg} at iteration level, with its own coefficients and a scale of 0.45; a rejected candidate receives a base reward of -0.08 before shaping.

D. LEARNED ACCEPTANCE CRITERION (LAC): SA WITH PREDICTIVE LOOKAHEAD OVERRIDE

In classical ALNS, move acceptance relies on Simulated Annealing (SA), which accepts improving transitions ($\Delta C < 0$) deterministically and non-improving transitions ($\Delta C \geq 0$) with probability $P_{\text{SA}} = \exp(-\Delta C/T)$. During deep cooling stages ($T \to 0$), P_{SA} vanishes exponentially, restricting search to downhill moves , so it can get stuck in local optima.

To reduce this trapping, the Learned Acceptance Criterion (LAC) adds an online supervised binary classifier $\phi_{\text{LAC}} : \mathbb{R}^9 \to [0,1]$. LAC is consulted only for feasible candidates that keep the current fleet size and increase travel distance; improving, fleet-reducing and infeasible candidates follow the rules of Algorithm 1 unchanged, and penalty-relaxed phases use the penalized SA rule without LAC. For these candidates, LAC does not filter or replace SA. It adds a disjunctive rule:

$$\begin{aligned} \text{Accept}(s' \mid s, T) \iff & \Big(p_t \geq \tau_{\text{acc}}(t)\Big) \\ & \vee \Big(P_{\text{SA}}(\Delta C, T) > U\Big) \end{aligned} \tag{19}$$

where $p_t = \phi_{\text{LAC}}(\mathbf{z}_t)$, $U \sim \text{Uniform}(0,1)$, and $\tau_{\text{acc}}(t) \in [0.40, 0.75]$ is an adaptive confidence threshold.

The disjunction ($\vee$) means LAC can only add acceptances. Any consulted move that standard Simulated Annealing would accept ($P_{\text{SA}} > U$) is also accepted by rule (19). When SA would reject a non-improving move ($P_{\text{SA}} \leq U$) , the move is still accepted if the classifier predicts a later incumbent improvement with probability $p_t \geq \tau_{\text{acc}}(t)$. These extra transitions mean that rule (19) defines a different Markov transition kernel from SA, and the classical asymptotic convergence results for SA under logarithmic cooling [41] do not carry over. We make no convergence claim for the modified kernel: the search uses geometric cooling within a finite budget, and the contribution of LAC is assessed empirically (Section V-E). The supervision target is observational. The first $W_{\text{lac}} = 300$ consulted decisions use pure SA while the classifier collects training data; for every consulted move, including warmup moves. the label records whether the global best travel distance improved within the next $H_{\text{lac}} = 80$ LAC decisions (not 80 ALNS iterations), irrespective of whether the move was accepted. The label therefore indicates a promising search context rather than the causal effect of accepting that particular move.

Because rule (19) only adds acceptances, it increases the number of accepted non-improving moves during deep cooling. Sections V-E and VI-C return to the difficulty of separating useful lookahead from a higher acceptance rate.

1) Features and Training

The LAC input feature vector $\mathbf{z}_t \in \mathbb{R}^9$ comprises 9 normalized features:

$$\begin{aligned} \mathbf{z}_t = \Big[& \frac{\Delta C}{|C_{\text{cur}}|},\ \frac{T}{T_0},\ \frac{\text{no_imp}}{\text{patience}}, \\ & \text{clip}(\Delta N_{\text{V}}, -2, 2),\ \min\Big(\frac{t}{10000}, 1\Big), \\ & \tau_{\text{tight}},\ \rho_{\text{fill}},\ \bar{s},\ \exp\Big(-\frac{\max(\Delta C, 0)}{T}\Big)\Big] \end{aligned} \tag{20}$$

During the first 300 consulted decisions, the solver uses Simulated Annealing alone while queuing the features of each decision. After a further $H_{\text{lac}} = 80$ consulted decisions, each queued decision receives a binary label: $y_t = 1$ if the best distance C_{best} has decreased since that decision, and 0 otherwise. The label therefore follows distance, not the lexicographic objective, so a fleet reduction that raises the best distance receives $y_t = 0$. The threshold rises with progress $p = \min(t/10000, 1)$ as $\tau_{\text{acc}}(t) = 0.40 + 0.35\, p^{0.75}$.

Because training examples are labeled online from the active search trajectory, the empirical training distribution is endogenous and non-stationary. Every 20 consulted decisions, LAC trains a 4-layer MLP ($9 \to 64 \to 48 \to 32 \to 1$, with LayerNorm and 0.1 dropout after the first layer) on a batch of 64 from a buffer of 5,000 labeled decisions, using Adam ($\text{lr} = 10^{-3}$) with class-weighted binary cross-entropy loss:

$$\mathcal{L}_{\text{LAC}}(\phi) = -\frac{1}{B} \sum_{i=1}^{B} \Big[w_1 y_i \log p_i + (1 - y_i) \log(1 - p_i) \Big] \tag{21}$$

where $p_i = \phi_{\text{LAC}}(\mathbf{z}_i)$ and $w_1 = N_{\text{neg}}/N_{\text{pos}}$ balances positive and negative labels within the batch.

E. HEURISTIC SEARCH COMPONENTS

The low-level search combines cached insertion checks, granular candidate lists, a frozen GNN heatmap and ejection chains.

1) Constant-Time Insertion Feasibility Queries via Savelsbergh Forward Slack

To avoid an $O(m)$ recomputation of route timing for each candidate insertion, we maintain Savelsbergh forward time slack F_i for each node i along route $R = (v_0, v_1, \ldots, v_m)$ [42]:

$$F_i = \min_{i \le j \le m} \left\{ l_j - w_j + \sum_{k=i+1}^{j} W_k \right\} \tag{22}$$

A candidate insertion of customer u between positions i and $i+1$ inducing arrival delay Δt_u at node $i+1$ preserves temporal feasibility for all subsequent customers if and only if $\max(0, \Delta t_u - W_{i+1}) \le F_{i+1}$, where W_k is the waiting time at node k, given that the prefix up to v_i is feasible and u is served within its window; capacity feasibility additionally requires $q(R) + q_u \le Q$. Given a cached timing profile of the route (arrival times, load, and forward slacks or, equivalently, the latest feasible service start of each node, which our implementation stores), each such query costs $O(1)$. The arrival time delay shifted to customer v_{i+1} is computed as:

$$\Delta t_u = \max\big(0, e_u - (w_i + s_i + t_{i,u})\big) + t_{i,u} + s_u + t_{u,i+1} - t_{i,i+1} \tag{23}$$

where w_i is the service start time at node i, s denotes service durations, and e_u is the earliest ready time. The $O(1)$ bound applies to a single insertion query and holds only while the cached profile is valid. Building or refreshing the profile costs $O(m)$ for a route of length m and is required after every accepted modification of that route; scanning all $m+1$ insertion positions of one customer is therefore $O(m)$ per route, and evaluating one customer against K routes is $O(Km)$ before granular pruning. Moves that reverse or relocate a segment (2-opt, ejection chains) recompute arrival times along the modified segment, so their cost grows with the segment length, and only the suffix check after the segment uses the cached slack. We claim no $O(1)$ bound for complete moves or iterations.

2) Granular Correlated Neighborhood Search (GNS)

Following granular search principles [43], inter-route operators restrict candidate evaluations to the $k = 15$ nearest customers of each customer by travel distance, precomputed once per instance with the depot excluded. This bounds the candidate insertion search space to $O(kN)$ move queries per iteration.

3) Lightweight Graph Neural Network for Spatial Edge Guidance

A lightweight Graph Neural Network (GNN) adds a learned edge probability heatmap $\mathbf{H} \in [0,1]^{(N+1)\times(N+1)}$ to the heuristic candidate lists. The graph encodes $N+1$ nodes (depot + customers) with 6-dimensional normalized node features $\mathbf{v}_i \in \mathbb{R}^6$:

$$\mathbf{v}_i = \left[\tilde{x}_i,\ \tilde{y}_i,\ \frac{q_i}{Q},\ \frac{e_i}{T_{\text{depot}}},\ \frac{l_i}{T_{\text{depot}}},\ \frac{s_i}{T_{\text{depot}}} \right] \tag{24}$$

where $(\tilde{x}_i, \tilde{y}_i) \in [0,1]^2$ are min-max normalized Cartesian coordinates, q_i/Q is demand normalized by vehicle capacity, and (e_i, l_i, s_i) are ready time, due time, and service duration normalized by the depot operational horizon T_{depot}. Message passing runs over sparse k-nearest-neighbor edge sets ($K = 16$, with slot 0 pinned to the depot) with 1-dimensional edge feature $e_{ij} = c_{ij}/c_{\max}$ representing normalized Euclidean distance.

The network architecture comprises a linear embedding layer ($d_h = 64$), 3 sparse joint node-edge message-passing layers with LayerNorm and residual connections, and a bilinear projection head:

$$\mathbf{H}_{ij} = \sigma\left(\mathbf{z}_i^\top \mathbf{W}_{\text{edge}} \mathbf{z}_j\right) \tag{25}$$

producing edge selection probabilities $\mathbf{H}_{ij} \in [0,1]$. Message passing uses the 16 nearest neighbors of each node. The registered checkpoint was trained offline with a weighted binary cross-entropy loss and sampled negative edges on 48 synthetic instances (16 each of the C, R and RC distributions, half with $N = 100$ and half with $N = 200$), with 12 further synthetic validation instances of the same sizes. Training targets are the route edges of a 10-second ALNS run without the GNN, so they are heuristic labels and not optimal edges. The checkpoint was selected by minimum validation loss (epoch 71 of 150, Adam, learning rate 10^{-3}, one training seed), and no benchmark test instance was used for training or selection. On the 12 validation instances, thresholds of 0.05, 0.01 and 0.003 retain 23.3%, 29.6% and 34.1% of candidate arcs and 99.45%, 99.90% and 99.90% of teacher edges. These instances also selected the checkpoint, so the figures are optimistic, and they measure agreement with heuristic labels, not a routing gain. On the 74 benchmark instances at threshold 0.01, the heatmap retained 34.0%, 23.3% and 23.6% of candidate arcs at 100, 200 and 400 customers and kept 99.1%, 99.8% and 100.0% of the route edges in the PyVRP-ILS solutions of the rerun at 300 s, and 98.0%, 99.2% and 99.8% of those of NeuroLKH-adapted. The lowest figure over the three thresholds was 96.6%, for NeuroLKH-adapted at 100 customers and threshold 0.05. These figures measure agreement with the routes of other solvers, which are themselves heuristic, and not a routing gain. The effect of the GNN on solutions is estimated only by the factorial ablation. $N = 400$ lies outside the training sizes, so use at that size is an extrapolation. During search, network weights are frozen, and inference is executed once per instance during preprocessing in $O(NK)$ time, populating a static heatmap $\mathbf{H}$ that guides repair and filters insertion positions.

4) Generalized Ejection Chains (GEC)

To attack vehicle-count plateaus , a 3-level ejection chain tries to eliminate the shortest route. It is attempted at every iteration in mode a_5, every fourth iteration when a route has at most four customers, and every twentieth consecutive iteration without improvement. Customers of the target route are processed by decreasing demand, and each displaced customer u triggers a search over all other routes:

(i) Level 1 (Direct Insertion): insert u into another route R_j at minimum added distance;
(ii) Level 2 (Single-Customer Displacement): eject $v \in R_j$, insert u into R_j, and insert v into a third route R_k; and
(iii) Level 3 (Two-Customer Chained Displacement): eject $v \in R_j$ and insert u; eject $w \in R_k$ and insert v; and insert w into a further route. This level runs only when at most 30 routes remain.

The search has no candidate beam and no internal time check, so the solver can stop it only after it returns, while the supervisor enforces the cutoff independently. Each feasibility test replays the modified route at linear cost, not the $O(1)$ cached query above. For m displaced customers, K routes and maximum route length L, the worst-case work is $O(mK^3L^4)$ (Supplementary Section 3).

5) Feasibility Invariant Across Operational Layers

Feasibility differs by operational level: (1) Current solution (s_{cur}): may be infeasible under penalized acceptance (Section IV-B3); (2) Candidate moves (s_{cand}): violations are penalized in those phases and rejected otherwise; (3) Best solution (s_{inc}): only solutions with full capacity and time-window compliance can update it; (4) Elite archive and (5) Route pool: hold only feasible solutions and routes; and (6) Returned solution (s^*): counted only after the supervisor validates it in the original problem space (Section V-A).

F. MATHEURISTIC RECOMBINATION: TIME-LIMITED HIGHS SET PARTITIONING

During search stagnation, elite feasible routes accumulated in Route Pool $\mathcal{P}$ are recombined by solving a time-limited Set Partitioning integer linear programming formulation:

$$\min \quad \sum_{r\in\mathcal{P}} (\lambda_{\text{pen}}\bar{c} + c_r)\, y_r \tag{26}$$

$$\text{s.t.} \quad \sum_{r\in\mathcal{P}:i\in r} y_r = 1, \quad \forall i \in \mathcal{N} \tag{27}$$

$$y_r \in \{0,1\}, \quad \forall r \in \mathcal{P} \tag{28}$$

where c_r is route travel distance, $\bar{c}$ is pool mean route distance, and $\lambda_{\text{pen}} \in \{2.0, 5.0, 12.0, 30.0, 50.0\}$ is a penalty multiplier; the vehicle penalty $\lambda_{\text{pen}}\bar{c}$ has a floor of 200 in the implementation. At most 800 columns of the pool enter one model, and a vehicle ceiling constraint may be added. If exact partitioning fails, the solver falls back to a set-covering relaxation with duplicate customers removed afterwards.

Proposition 1 (Penalty Bound for Lexicographic Fleet Preservation): *Let $\mathcal{Y}_{SP} \subseteq \{0,1\}^{|\mathcal{P}|}$ be the set of feasible partitions over route pool $\mathcal{P}$. If $\lambda_{pen}\bar{c} > \sup_{\mathbf{y}_1,\mathbf{y}_2\in\mathcal{Y}_{SP}} \left(\sum_{r\in\mathbf{y}_1} c_r - \sum_{r\in\mathbf{y}_2} c_r\right)$, any partition $\mathbf{y}_1$ with $\sum_r y_{1,r} < \sum_r y_{2,r}$ strictly achieves a lower objective value than $\mathbf{y}_2$, guaranteeing exact lexicographic fleet minimization priority over distance within the column subset $\mathcal{P}$.*

With Route Pool $\mathcal{P}$ limited to $|\mathcal{P}| \le 2000$ columns, the distance difference between two valid partitions is bounded by $K_{\max} \cdot c^{\text{route}}_{\max}$. To satisfy Proposition 1 at every size, including $N = 400$, the solver applies the certified threshold $\lambda^{\text{cert}}_{\text{pen}} = \max\left(\lambda_{\text{pen}}\bar{c},\ (1+\epsilon_{\text{margin}})K_{\max}c^{\text{route}}_{\max}\right)$ with clearance margin $\epsilon_{\text{margin}} = 0.20$ at the largest scale of the multi-scale penalty grid $\lambda_{\text{pen}} \in \{2.0, 5.0, 12.0, 30.0, 50.0\}\bar{c}$ (formal derivation in Supplementary Material Proposition 1) . The smaller scales are tried first and do not, in general, satisfy the condition of Proposition 1, so the partition they return need not be lexicographically optimal within $\mathcal{P}$; every partition is, however, subject to the hard fleet ceiling of the query. Proposition 1 therefore concerns lexicographic ordering among partitions of the route pool at the certified scale only. It does not imply optimality of the partition for the original VRPTW, whose optimal routes need not be in $\mathcal{P}$.

Across the whole search, the hierarchical objective is enforced at the level of the incumbent rather than at every transition. The current solution may temporarily accept a one-vehicle increase in the diversification mode, and penalty-relaxed phases may visit infeasible states. However, a candidate replaces the best solution only if it has fewer vehicles, or the same number of vehicles and strictly lower distance, after independent validation of exact cover, capacity and time windows in the original instance. Consequently the reported solution is non-increasing in N_{V} over time and never trades a vehicle for distance, but no claim is made that every intermediate move respects the ordering.

1) Route Pool Deduplication and Capacity Management

Routes visiting identical customer sets $\text{cover}(R) = \{v_1, \ldots, v_m\}$ are deduplicated with hash tables, which keep only the shortest tour. Pool capacity $|\mathcal{P}|_{\max} = \min(2000, 600 + 4N)$ is partitioned into Slot A (75% capacity, sorting by cost-per-stop $c_r/|r|$) and Slot B (25% capacity, preserving longest sub-routes $|r|$ to supply high-occupancy columns for fleet reduction), with at most $M_{\text{cust}} = \min(60, 28 + \lfloor N/10 \rfloor)$ routes per customer. HiGHS [44] solves the models by branch and cut. Each penalty scale is one query, limited to $\max(1, \min(30, 4 + 0.05N)/3)$ s (3, 4.7 and 8 s at $N = 100, 200$ and 400) and capped by the remaining budget.

G. GUIDED EJECTION SEARCH PHASE

An optional phase runs the route-minimization heuristic of Nagata and Bräysy [20] once, before the main search, for 10% of the remaining budget. It removes a random route and places its customers in an ejection pool. It then takes customers from the pool one at a time and inserts each one if a feasible insertion exists. Otherwise it increments the

Algorithm 1 Tri-Level Hybrid DDQN-ALNS Overview

Require: Instance $\mathcal{I}$, Hyperparameters Θ, Time budget T
Ensure: Best feasible routing plan s^*
$s^* \leftarrow s_{\text{inc}} \leftarrow \text{GreedyInit}(\mathcal{I})$; $\mathcal{P} \leftarrow \text{ExtractRoutes}(s^*)$
$s^* \leftarrow s_{\text{inc}} \leftarrow \text{GuidedEjectionSearch}(s^*)$ for 10% of the remaining budget ▷ Classical, Section IV-G
Initialize Macro DDQN Q_{macro}, Micro DDQN Q_{micro}, LAC ϕ_{LAC}
for $t = 1, 2, \ldots$ until 80% of T has elapsed **do**
  if $t \bmod \Delta_{\text{seg}} == 1$ **then** ▷ Macro Tier: SMDP regime selection
    Update regime $a_k \leftarrow \pi_{\text{macro}}(\mathbf{s}_k^{\text{macro}})$; trigger GEC/HiGHS if selected
  end if
  Select $(d, r) \leftarrow \pi_{\text{micro}}(\mathbf{s}_t^{\text{micro}})$ via confidence gating (18)
  $s_{\text{cand}} \leftarrow \text{GranularLocalSearch}(\text{Repair}(\text{Destroy}(s_{\text{cur}}, d), r), k = 15)$
  Accept s_{cand} by the phase's rule: penalized SA, or SA or LAC (ϕ_{LAC})
  Update route pool $\mathcal{P}$, train DDQNs and LAC on mature experiences
end for
Run tail phases until T: recombination, route elimination, polishing
return s^* ▷ Certified by the supervisor

customer's penalty counter, applies the feasible ejection of at most $k_{\max} = 5$ customers from one route that has the smallest penalty sum, and perturbs the solution with 100 random feasible relocate, exchange and 2-opt* moves. A route is eliminated when the pool empties, and the attempt restarts after 20,000 steps otherwise. The phase ends at its time limit or at the capacity bound $\lceil \sum_i d_i / Q \rceil$, and it reads no reference fleet. If it reduces the fleet, the result receives one local-search pass before the main search. This is a classical method, so any gain from it is not a gain from learning.

V. EXPERIMENTAL EVALUATION

This section evaluates the method on 74 standard benchmark instances: Solomon-100 (56 instances across 6 families) [2], Gehring-Homberger 200-customer (12 instances) and Gehring-Homberger 400-customer (6 instances) [45]. Five questions structure the evaluation:

- RQ1 (Comparison at equal compute): At 300 s, how does the method compare with ALNS-Base, PyVRP-ILS and NeuroLKH-adapted on fleet size and, at matched fleet size, on distance?
- RQ2 (Budget): How do these comparisons change at 10 and 60 s?
- RQ3 (Components): Which components change results, and does the learning bundle add to the heuristic bundle?
- RQ4 (Scale): How do the comparisons differ at 100 , 200 and 400 customers?
- RQ5 (Families): How do they differ across clustered, random and mixed instance families? This question is descriptive.

A. EXPERIMENTAL SETUP, BASELINE FAIRNESS, AND COLD-START PROTOCOL

All experiments ran on one Apple M1 notebook (4 performance and 4 efficiency cores , 16 GB memory, macOS 26.5.2), with four cells at a time and one numerical-library thread per cell. macOS assigns cores itself, so processes were not pinned to cores.

Machine-speed variation. A wall-clock budget does not fix the compute a run receives. PyVRP and NeuroLKH with the same seed retrace the same search, so the time needed to reach the same (fleet, distance) step measures machine speed. On 18 instances at 60 s, cells of our first full sweep took a median of 2.3 times as long as the same cells on a later night, ranging from about 0.65 to 9.6 times per instance. During that sweep the notebook was also used for other work and was at times unplugged or asleep, but the cause of the slowdown is not established. Absolute times therefore do not describe a steady machine speed, and comparisons are valid within a block of cells that ran together, not across blocks. In the reported rerun, all cases of one instance, seed and cutoff run together as a block. A cell whose wall-clock span exceeds its monotonic duration by more than 5 s was frozen by machine sleep, and its whole block runs again. Each cell records power state and load at its start, and the primary comparisons are repeated without blocks that started a cell at a one-minute load above 8. Other work on the notebook raised the load as high as 233 early in the rerun. Before any result was read, we therefore moved 58 of the first 111 blocks at 300 s, those with a cell started above load 8, aside unread and ran them again, and from then on no new cell started above that load. A cell can still start below that load and then run above it, so also before any result was read, we began sampling load once a minute and added two repeats at 300 s: one without blocks that have a cell whose mean load during its run was above 8, and one that also leaves out blocks with a cell not measured this way. Load during a run is not known for the roughly 450 cells at 300 s that finished before sampling began.

In all, 11 blocks were rerun after machine sleep and 58 after load spikes. The load repeats change none of the three primary comparisons, but for the adoption rule the interval then includes zero (Section V-C).

ALNS-Base is our own implementation in the framework of Ropke and Pisinger [5], [6], not a reproduction of it. It shares the 13 destroy and 5 repair operators of our search infrastructure. It loads no GNN, so its two heatmap-guided destroy operators receive no heatmap. Pairs are selected by Thompson sampling over 65 Beta posteriors instead of a roulette wheel, with scores $(\sigma_1, \sigma_2, \sigma_3) = (33, 9, 3)$ normalized by σ_1, posterior decay 0.95 every 100 iterations, and a posterior reset after 1,250 iterations without improvement. Simulated Annealing starts at $T_0 = 0.05\, c_0 / \ln 2$, where c_0 is the cost of the starting solution, and cools geometrically with $\alpha = 0.99975$. Its main loop applies no local search, ejection chains or set partitioning. Initialization is algorithm-specific: ALNS-Base starts from the unweighted greedy construction (BuildGreedyInitialSolution) , the hybrid starts from the same construction with GNN-weighted insertion costs, and PyVRP-ILS and NeuroLKH use their own construction procedures; construction and model inference are charged to each solver's budget. A defect in the greedy construction, which did not check the depot return deadline, made the ALNS-Base start infeasible on R1_2_5, so all 20 ALNS-Base runs on that instance terminated without a solution. We corrected the construction, which leaves the start unchanged

on the other 73 instances, and reran those 20 runs.

PyVRP-ILS is the iterated local search of PyVRP [46], version 0.14.1, with a vehicle fixed cost set above any possible travel cost so that its objective follows the fleet-first order. PyVRP's HGS is not the method used here. NeuroLKH-adapted runs the pretrained CVRPTW checkpoint and LKH binary of NeuroLKH [47] at a pinned upstream commit, with cold restarts at decreasing fleet sizes. It is an adaptation, not a replication of the authors' experiments, and we did not retrain it or tune it on benchmark instances. Both baselines work on integer-scaled models, so every candidate is mapped back and certified in the original problem space.

All cases share one supervisor. A case's clock starts in a fresh process with an empty work directory and JIT cache, and construction, model inference, search, JIT compilation after the start, certification and communication with the supervisor are charged to its budget. Imports and loading of frozen models are recorded separately. An incumbent counts only if the supervisor has received, validated and hashed it before the cutoff, and the process is expected to return within $[T - 0.05, T + 0.5]$ s of the cutoff T. A later return is recorded but does not change which incumbent counts (Section V-G).

a: Reference-Fleet Information

In the first full sweep (Section V-I), the hybrid read the best-known fleet size $N_{\mathrm{V,BKS}}$ [48] of each benchmark instance during search. It served as the floor at which vehicle-reduction phases stop, as the trigger for switching set partitioning to a distance-only objective, and as a parameter of the recombination and route-elimination steps. None of the baselines uses this information, it is not available for new instances, and equal compute alone does not remove this asymmetry. The rerun therefore disables every lookup. With the Guided Ejection Search phase on, the capacity bound takes the place of the reference fleet; with it off, the hybrid uses its fallbacks for unknown instances. The effect of reading the reference fleet was not re-estimated under matched conditions, so we report only that the first sweep's hybrid read it.

b: Hyperparameter Specification and Experimental Firewall

The hyperparameters in Table 3 combine literature defaults (for example the Adam optimizer parameters and the Polyak update rate $\tau = 0.005$) with values set by hand during development. We did not retain a calibration log, so we cannot document how the hand-set values were chosen or show that they are independent of earlier runs on the benchmark instances. They were frozen before the sweeps reported here. One solver rule reads family information from the benchmark's naming convention: the fleet-reduction search receives more iterations when an instance name begins with RC and the current fleet exceeds the target by at least one vehicle.

The test set did not, however, remain untouched by development. After the first full sweep, per-family losses on the test instances led us to evaluate a Guided Ejection Search phase before the main search. This is a classical method, so any gain from it is not a gain from learning. Its parameters were chosen only on 30 development instances with different instance identities: 18 are 100-customer subsamples of Gehring-Homberger instances, and 12 are unmodified Gehring-Homberger instances outside the test set. They are not independent of the test set, because they share customer coordinates with test instances of the same families and up to 81% of a development instance's customer records (coordinates, demand, time window and service time) are identical to those of a test instance. The rule for adopting the phase was fixed before a single contemporaneous rerun on the test set, and that rerun was not used to change the solver. The rule adopted the phase by a small margin (Section V-C).

Training and testing protocol. The Macro DDQN, Micro DDQN and LAC have no offline training phase. They start from random weights in every run and learn online within that run's time budget, so they have no train/test split, nothing learned in one run is carried to another, and we make no claim that a learned policy transfers between instances. Only the GNN is trained offline. It is then frozen and uses only the synthetic data described in Section IV-E3. Its generalization is limited to what the factorial ablation measures on the benchmark instances.

The protocol has four rules for keeping runs independent and reproducible:

1) Cold starts: each solver run uses its own temporary directory (`ColdStartScope`), deleted when the run ends, and the Macro DDQN, Micro DDQN and LAC start from random weights;
2) Deterministic RNG Seeding: each worker process seeds Python's `random`, `numpy`, and PyTorch (`torch.manual_seed` and `torch.cuda.manual_seed_all`), using 5 independent seeds ($\{42, 43, 44, 45, 46\}$);
3) Objective Hierarchy: fleet size is the primary endpoint, and distance is compared only between seed pairs with matched fleet size (Section V-B); and
4) Recorded configuration: every run writes JSON metadata with `resolved_components`, which lists the modules that were active.

B. STATISTICAL ANALYSIS

Estimands and intervals follow a design fixed before the sweeps. The unit is a seed pair within an instance, not a best run or an instance mean. For each pair, the *fleet score* is $+1$ if the proposed configuration uses fewer vehicles, -1 if it uses more, and 0 if they tie; a case with a solution beats a case without one, and two cases without a solution tie. Scores are averaged over seeds and instances within each size and combined with fixed weights of $56/74$, $12/74$ and $6/74$. The score therefore lies in $[-1, 1]$ and is the net share of seed pairs won. It is not a difference in mean vehicles.

TABLE 3. Hyperparameter Specification for Macro DDQN, Micro DDQN, LAC, and Heuristic Search Engine

Component	Hyperparameter	Value / Specification
Macro DDQN	Network Architecture	Dueling MLP: Input(12) → FC(128) → LayerNorm → FC(128) → LayerNorm → Modes(7)
	Optimizer	Adam (lr $= 3 \times 10^{-4}$, Cosine Annealing to 10^{-5})
	Buffer & Batch Size	Prioritized Replay: 20,000 transitions, Batch 64
	Discount Factor (γ)	0.95
	Target Update (τ_{macro})	Polyak soft update $\tau_{\text{macro}} = 0.005$
Micro DDQN	Network Architecture	MLP: Input(20) → FC(128) → LayerNorm → ReLU → FC(128) → (Value / Adv)
	Optimizer	Adam (lr $= 3 \times 10^{-4}$, Cosine Annealing to 10^{-5})
	Buffer & Batch Size	Prioritized Replay: 30,000 steps, Batch 64
	Discount Factor (γ)	0.97
	Blending ($\alpha_{\text{p}}, \beta_{\text{b}}, \alpha_{\text{u}}, c_{\text{ucb}}$)	0.55, 0.20, 0.35, 1.0
LAC Model	Architecture	MLP: Input(9) → FC(64) → LayerNorm → ReLU → Dropout(0.1) → FC(48) → ReLU → FC(32) → ReLU → FC(1) → Sigmoid
	Lookahead Horizon (H_{lac})	80 LAC decisions
	Warmup Period	300 consulted decisions (Simulated Annealing only)
	Acceptance Threshold	Adaptive $\tau_{\text{acc}}(t) \in [0.40, 0.75]$
	Loss Function	Class-Weighted Binary Cross-Entropy
Search Engine	Granular Neighbors (k)	15 nearest customers by travel distance
	Set Partitioning (λ_{pen})	Grid: $\{2.0, 5.0, 12.0, 30.0, 50.0\}\bar{c}$, floor 200
	Route Pool Limit ($\lvert\mathcal{P}\rvert_{\max}$)	$\min(2000, 600 + 4N)$ routes
	MIP Cutoff	Per query: 3, 4.7, 8 s at $N = 100, 200, 400$ (HiGHS)

The *matched distance gap* is the percentage distance difference relative to the comparator, computed only on seed pairs with equal fleet size and averaged in the same way. Negative values favor the proposed configuration. It is conditional on matched fleets and is not an unconditional treatment effect.

Uncertainty comes from a percentile bootstrap (10,000 replicates, fixed seed) that resamples the five seeds and, within each size, the four groups of instances that share customer geometry, so that instances with the same coordinates move together. With four groups per size the intervals are coarse, and several have an endpoint at exactly zero. An interval is reported as undefined when any replicate has no matched pair in a size stratum. The three primary comparisons at 300 s use Bonferroni-adjusted 98.33% intervals, and the eight primary factorial contrasts at 300 s use 99.38% intervals. Other budgets, size strata and exploratory contrasts use unadjusted 95% intervals and are descriptive.

A missing cell caused by infrastructure failure is not an algorithm outcome, so the primary treatment drops that seed pair from that comparison only, and we report bounds that count such gaps as failures. A cell in which the algorithm itself found no solution stays in the analysis and loses its pair.

C. COMPARISON AT 300 S (RQ1, RQ4)

Under the rule fixed before the rerun, the Guided Ejection Search phase was to be adopted only if the full configuration

TABLE 4. Rerun at 300 s: Proposed Configuration vs. Each Comparator (Intervals at 98.33%)

Comparator	Fleet score	Seed pairs W/T/L	Matched TD gap (%)
ALNS-Base	−0.027 [−0.065, 0.011]	4/352/14	+0.04 [−0.61, 0.66]
PyVRP-ILS	+0.008 [−0.098, 0.117]	37/299/34	+0.95 (undefined)
NeuroLKH-adapted	−0.068 [−0.142, 0.033]	16/313/41	−3.57 (undefined)

with all six components on and the phase (F111111) used fewer vehicles than the same configuration without the phase (F111111-blind) at 300 s on a 98.75% interval. It did: the fleet score was + 0.062 [0.001, 0.123], with 26 wins, 341 ties and 3 losses over seed pairs. F111111 is therefore the proposed configuration and is compared with ALNS-Base , PyVRP-ILS and NeuroLKH-adapted below. The margin is small. In the repeat fixed before results were read, which leaves out the 92 blocks with a cell run at a mean load above 8, the estimate stays at + 0.062 but the interval becomes [−0.014, 0.123] and no longer excludes zero. On blocks whose load was measured throughout, the interval is undefined (point estimate +0.054). Results from the first sweep on the test instances motivated adding this phase, so the outcome is not an independent test of that design choice. The phase is classical, so a gain from it is not evidence for learned control, and because every ablation case keeps it on, the ablation cannot apportion the gain to it.

Table 4 gives the three primary comparisons at 300 s. The proposed configuration did not differ detectably on fleet size from ALNS-Base (−0.027 [−0.065, 0.011]), PyVRP-ILS (+0.008 [−0.098, 0.117]) or NeuroLKH-adapted (−0.068 [−0.142, 0.033]). Most of the 370 seed pairs tied: 352 against ALNS-Base , 299 against PyVRP-ILS and 313 against NeuroLKH-adapted. Where fleets matched, the distance gap against ALNS-Base was +0.04% [−0.61, 0.66]. The gaps against PyVRP-ILS (+0.95%) and NeuroLKH-adapted (−3.57%) have undefined intervals, so neither is given a direction.

By size, on unadjusted 95% intervals, no comparison excluded zero. The largest point estimates were against PyVRP-ILS at 200 and 400 customers (+0.233 [0.000, 0.386] and +0.300 [0.000, 0.750]), against ALNS-Base at 400 customers (−0.233 [−0.486, 0.000]) and against NeuroLKH-adapted at 200 customers (−0.167 [−0.250, 0.000]). With four geometry groups per size the bootstrap is coarse, and each of these intervals ends exactly at zero, so none shows a difference. No cell was lost to infrastructure failure, so the three ways of treating gaps give identical results, and no block started at a load above 8. The repeats based on load during a run change no conclusion. Leaving out the 92 blocks with a cell run at a mean load above 8 gives −0.030 [−0.080, 0.015], +0.007 [−0.109, 0.122] and −0.070 [−0.158, 0.044]. Also leaving out the 76 blocks whose load was not measured throughout leaves the intervals undefined.

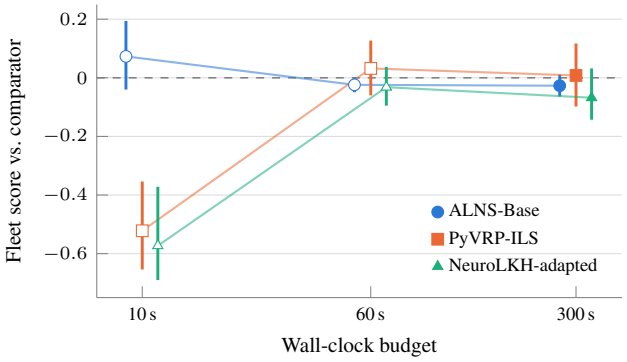


FIGURE 2. Fleet score of the proposed configuration against each comparator by wall-clock budget. Positive values favor the proposed configuration. Filled markers at 300 s show the confirmatory 98.33% intervals; open markers at 10 and 60 s show descriptive, unadjusted 95% intervals. Markers are offset horizontally for legibility.

D. EFFECT OF THE TIME BUDGET (RQ2)

Fig. 2 shows how the comparisons change with the budget. These results are descriptive and use unadjusted 95% intervals. At 10 s the proposed configuration used more vehicles than PyVRP-ILS (-0.522 [-0.654, -0.354]; 16 wins, 145 ties and 209 losses) and NeuroLKH-adapted (-0.573 [-0.690, -0.372]; 14, 130 and 226), and it did not differ detectably from ALNS-Base ($+0.073$ [-0.040, 0.194]). At 60 s it did not differ detectably from any of the three: -0.024 [-0.049, 0.000] against ALNS-Base, $+0.032$ [-0.060, 0.128] against PyVRP-ILS and -0.032 [-0.095, 0.036] against NeuroLKH-adapted. The interval against ALNS-Base ends exactly at zero, so it does not show a difference. Where fleets matched at 60 s, the distance gap against ALNS-Base was $+0.03\%$ [-0.54, 0.49]. The other matched gaps at 10 and 60 s have undefined intervals or no matched pairs and are given no direction.

Compilation of the search kernels and the Guided Ejection Search phase are charged to the budget. In the 10 s cells, the first improvement from any later phase came no earlier than 5.2 s, with a median of 6.2, 6.3 and 7.3 s at 100, 200 and 400 customers. In 123 of the 370 cells, no later phase improved on the solution the Guided Ejection Search phase left. Most of the first half of the budget therefore passed before the main search improved anything, so the 10 s comparison says little about the search itself.

The 900 s stage was started and not completed, and no result from it is reported.

E. FACTORIAL ABLATION (RQ3)

The ablation uses 18 instances (the first instance of each of the six families at each size), five seeds, and budgets of 60 and 300 s. Each of six factors (GEC, SP, Macro, Micro, LAC, GNN) is set to its low level in the full configuration and to its high level in the bare configuration. Low levels are not simple removals: GEC keeps direct insertion, SP disables every recombination query, Macro and Micro use rule-based policies in place of the DDQNs, LAC uses Simulated Annealing alone, and GNN runs without a heatmap. A 2×2 design contrasts the learning bundle (Macro, Micro, LAC, GNN) with the heuristic bundle (GEC, SP). The case with all six factors low shares the search infrastructure, including the tail phases, and is not ALNS-Base. In the rerun the Guided Ejection Search phase is a fixed layer in all cases, so effects are measured on top of it.

TABLE 5. Factorial Ablation at 300 s: Primary Contrasts (Intervals at 99.38%)

Contrast	Fleet score	Matched TD gap (%)
GEC full context	$+0.000$ [-0.110, 0.072]	-0.17 [-1.22, 0.54]
SP full context	-0.044 [-0.181, 0.067]	$+0.25$ [-1.17, 2.01]
Macro full context	-0.022 [-0.120, 0.043]	-0.27 [-1.60, 0.71]
Micro full context	$+0.044$ [-0.095, 0.201]	-1.29 [-3.98, 0.57]
LAC full context	-0.022 [-0.098, 0.000]	-0.32 [-1.08, 0.22]
GNN full context	-0.044 [-0.167, 0.067]	$+0.24$ [-1.08, 1.84]
heuristic at no learning	-0.044 [-0.216, 0.067]	$+0.52$ [-0.50, 1.76]
learning at heuristic	$+0.022$ [-0.111, 0.152]	-0.08 [-1.55, 1.86]

Positive fleet scores and negative gaps favor the component or bundle named. All 32 contrasts are in the supplement.

Table 5 gives the eight primary factorial contrasts at 300 s. No primary factorial contrast excluded zero at 300 s, so the ablation does not establish a benefit of the learned components at that budget. Most seed pairs tied, between 80 and 88 of the 90 in every primary contrast, and the point estimates ranged from -0.044 to $+0.044$. The learning bundle with the heuristic bundle present gave $+0.022$ [-0.111, 0.152], and Micro in the full context gave $+0.044$ [-0.095, 0.201].

The LAC contrast is uninformative rather than evidence that LAC has no effect. Its fleet score was -0.022 [-0.098, 0.000], with no wins, 88 ties and 2 losses. In a separate one-seed measurement on the same 18 instances at 300 s (Section V-H), LAC was consulted in 6 instances and passed its 300-decision warmup in 3, and its classifier did not approve a move in any of them. At this budget, acceptance therefore followed the Simulated Annealing rule throughout. In the same measurement, the macro controller made no decision on the six R2 and RC2 instances.

At 60 s, on unadjusted 95% intervals, three descriptive contrasts excluded zero: Micro in the bare configuration ($+0.344$ [0.198, 0.516]), Micro in the full context ($+0.244$ [0.053, 0.492]) and the learning bundle without the heuristic bundle ($+0.256$ [0.114, 0.433]). They describe a pattern and do not establish one, and 47% of the 60 s ablation cells ran at a mean load above 8, measured with about one load sample per cell. At 300 s, 56% of ablation cells ran at a mean load above 8. No load repeat was fixed in advance for the ablation, so this exposure is reported but not adjusted for.

The pattern for Micro-DDQN at 60 s came from exploring results on the test set, so we tested it once more afterwards, under a protocol frozen before any cell ran. The size breakdown had been read first and motivated using 200 and 400 customers only. The test used Homberger instances 9 and 10 of each family at both sizes (24 instances), which are new time-window variants of the layouts of the ablation instances and not new layouts, with the four cases F111111, F111011,

F000100 and F000000, five seeds and 60 s; all 480 cells returned a valid solution. On the two confirmatory contrasts (97.5% intervals, sizes weighted equally), Micro-DDQN used fewer vehicles than the rule-based policy at 60 s on these instances in the bare configuration (+0.208 [0.025, 0.450]; 29 wins, 87 ties and 4 losses). In the full context the test did not detect a difference and is too small to rule one out (+0.150 [−0.012, 0.372]; 25 wins, 88 ties and 7 losses). On descriptive 95% intervals by size, the bare contrast was +0.133 [−0.043, 0.380] at 200 customers and +0.283 [0.029, 0.640] at 400, and the full-context contrast was +0.083 [−0.080, 0.386] and +0.217 [0.000, 0.460]. Where fleets matched, distance with Micro-DDQN was 11.7% lower in the full context (97.5% interval [−17.4%, −7.3%], 88 seed pairs) and 10.9% lower in the bare configuration, where that interval is undefined. Of the 120 blocks, 71 ran at a mean load above 8; without them the fleet scores were +0.114 (bare) and +0.157 (full context) and both intervals are undefined, so we state no direction for those blocks alone. The test covers 200 and 400 customers at 60 s and compares Micro-DDQN with a rule-based policy inside the same framework; it says nothing about 100 customers, about 300 s, or about ALNS-Base, PyVRP-ILS and NeuroLKH-adapted, and the 300 s results stand as reported.

F. INSTANCE FAMILIES (RQ5)

By family, the comparisons are descriptive: they count the seed pairs in which the proposed configuration used fewer, the same or more vehicles than each comparator at 300 s. The counts for every family and comparator are in Supplementary Section 8, with the per-instance means at each budget. On C1 and C2, every seed pair tied against all three comparators. Against ALNS-Base, the other four families were close, with 0 to 2 pairs fewer and 1 to 5 pairs more in each. Against PyVRP-ILS, the proposed configuration used fewer vehicles in 18 pairs and more in 1 on RC1, 10 and 10 on R1, 9 and 13 on R2, and none and 10 on RC2. Against NeuroLKH-adapted, the counts were 8 and 8 on R1, 8 and 2 on RC1, none and 21 on R2, and none and 10 on RC2. The losses therefore concentrate on R2 and RC2, the families with wide time windows: there the proposed configuration never used fewer vehicles than NeuroLKH-adapted, nor than PyVRP-ILS on RC2. Where fleets matched on R2 and RC2, its distance was 10.1% and 15.4% shorter than that of NeuroLKH-adapted, but no interval is computed per family, and under the lexicographic objective the fleet result comes first.

G. AVAILABILITY AND FEASIBILITY

The supervisor validates every incumbent in the original problem space, covering customer coverage, capacity and time windows, before it counts. A cell without such an incumbent is reported as unavailable, and it is never replaced by a later or uncertified result. In the first sweep, 21 of the 8,620 cells were lost to infrastructure failures. In the rerun, all 8,060 cells of the five completed stages finished, so no comparison has an infrastructure gap. Five cells found no certified solution, all from comparators at short budgets: NeuroLKH-adapted on R1_4_1 (seeds 43 and 45) and RC1_4_1 (seed 45) at 10 s, and PyVRP-ILS on RC1_4_1 (seed 42) at 10 and 60 s. Every cell of the proposed configuration found one. Blocks with a cell frozen by machine sleep were moved aside unread and run again whole, 11 blocks with 74 cells, and 58 blocks at 300 s (345 cells) were moved aside unread and rerun after load spikes on the machine. Of the 8,060 cells, 350 returned more than 0.5 s after their cutoff, 349 of them NeuroLKH-adapted (median 0.77 s late, at most 1.20 s), but in every cell the counted incumbent had been captured before the cutoff.

H. COMPUTATIONAL COST

Each of the 18 ablation instances was run once more with the proposed configuration (seed 42, 300 s), with timers on its main components. These runs measure shares of the budget, not solution quality. Each started at a one-minute load average of 3 or below, and four 400-customer runs ended at a load between 11 and 18. Set partitioning took the largest share, a median of 53%, 28% and 22% of the budget at 100, 200 and 400 customers. Destroy, repair and local search together took a median of 18%, 48% and 50%, and the Guided Ejection Search phase took its allotted 10%. The learned controllers were cheap: macro, micro and LAC inference and training together took a median of 2.6%, 1.9% and 0.8%, and at most 23%, on C101, where the main loop ran 213 iterations per second. GNN inference took 0.01% and certification of incumbents less than 0.05%. About 11% of the time was not attributed to a timed component. The main loop ran at a median of 17.8, 14.5 and 6.5 iterations per second at 100, 200 and 400 customers. The compute of the learned layer is therefore unlikely to explain the null results above. The activity counts in Section V-E suggest instead that parts of it rarely act at this budget. ALNS-Base was not measured in this way, so no iteration rate is reported for it.

I. RECORD OF THE FIRST SWEEP

The first full sweep is kept on record. Its hybrid read the reference fleet, had no Guided Ejection Search phase, and ran under the varying machine speed described in Section V-A, so its results are valid within blocks and do not supersede the rerun.

Table 6 shows that at 300 s this hybrid did not differ detectably from ALNS-Base or PyVRP-ILS on fleet size and used more vehicles than NeuroLKH-adapted. Where fleets matched, the distance gap against NeuroLKH-adapted was −4.62%, but that interval is undefined, so it is given no direction. Against PyVRP-ILS, no size stratum differed detectably: the fleet score was −0.150 at 100 customers, with a 95% interval [−0.257, 0] that ends at zero, and +0.150 and +0.300 at 200 and 400 customers, with intervals that include zero. At 10 s it was behind all three comparators on fleet size. At 900 s, on the unadjusted interval, it was ahead of ALNS-Base (fleet score +0.135, [0.022, 0.201];

TABLE 6. First Sweep at 300 s: F111111, Which Read the Reference Fleet, vs. Each Comparator (Intervals at 98.33%)

Comparator	Fleet score	Seed pairs W/T/L	Instances W/T/L	Matched TD gap (%)
ALNS-Base	+0.076 [−0.010, 0.134]	38/ 320/ 10	16/ 54/ 4	−0.23 [−0.65, 0.11]
PyVRP-ILS	−0.065 [−0.183, 0.083]	35/ 275/ 59	8/ 48/18	+0.84 (undefined)
NeuroLKH-adapted	−0.176 [−0.284, −0.029]	8/286/73	2/50/22	−4.62 (undefined)

Instances W/T/L compares mean fleet over five seeds and is descriptive. Treating infrastructure gaps as failures changes no conclusion.

matched gap -0.41%, $[-0.70, -0.03]$) and still behind NeuroLKH-adapted (-0.134, $[-0.224, -0.021]$).

In its factorial ablation, no primary contrast had a fleet interval excluding zero at 300 s under the 99.38% correction. On unadjusted intervals, the Micro DDQN against its rule policy in the full configuration ($+0.200$, $[0.060, 0.366]$) and the learning bundle added to the heuristic bundle ($+0.189$, $[0.067, 0.363]$) excluded zero at 60 s, and the same contrast in the bare configuration did so at 300 s ($+0.111$, $[0.022, 0.178]$). The evidence for learned control in this sweep is therefore descriptive.

VI. DISCUSSION

This section interprets the results, states their practical meaning, and lists the limitations of the method and of the evidence.

A. WHERE THE METHOD DOES AND DOES NOT HELP

Against ALNS-Base, the proposed configuration gained nothing detectable on fleet size at any budget: the intervals at 10, 60 and 300 s all contain zero or end at it, and most seed pairs tied. This comparison does not isolate the learned layer, because ALNS-Base also lacks the local search, ejection chains and set partitioning of the hybrid; the ablation is the comparison that does.

Against PyVRP-ILS and NeuroLKH-adapted, it did not differ detectably at 300 s or 60 s, and at 10 s it used more vehicles than both on unadjusted intervals, and compilation and the Guided Ejection Search phase take most of the first half of that budget. By family, the losses concentrate on R2 and RC2, the families with wide time windows: there it never used fewer vehicles than NeuroLKH-adapted, nor than PyVRP-ILS on RC2. It used fewer vehicles than PyVRP-ILS in 18 seed pairs and more in 1 on RC1, which is descriptive and does not establish an advantage.

The classical Guided Ejection Search phase matters more to ALNS-Base than to the hybrid: at 300 s, with the phase against without it, the fleet score was $+0.208$ for ALNS-Base but only $+0.062$ for the hybrid. On descriptive 95% intervals, ALNS-Base with the phase used fewer vehicles than without it ($+0.208$ $[0.052, 0.291]$; 80 wins, 287 ties and 3 losses). Without the phase, the hybrid used fewer vehicles than ALNS-Base ($+0.116$ $[0.002, 0.181]$) and more than NeuroLKH-adapted (-0.135 $[-0.236, -0.012]$); with the phase on both, the difference from ALNS-Base was no longer detectable. The hybrid's other classical components differ from ALNS-Base as well, so even the comparison without the phase is not evidence for learned control.

The ablation does not establish a benefit of the learned components at 300 s. At that budget the LAC classifier never changed an acceptance decision and the macro controller made no decision on the R2 and RC2 instances, so parts of the learned layer were hardly exercised; the LAC contrast is uninformative, not a null effect. The one signal for learning concerns Micro-DDQN at 60 s. In a separate test on new time-window variants of the same layouts, it used fewer vehicles than the rule-based policy in the bare configuration, the test did not detect a difference in the full context, and the bare result is not confirmed without the blocks run at high load (Section V-E). These conclusions concern the 74 benchmark instances, the 24 time-window variants of the Micro test, the budgets and the single machine of this study, and do not extend beyond them.

B. MANAGERIAL IMPLICATIONS FOR COMMERCIAL LOGISTICS

In commercial freight operations, capital depreciation, vehicle financing, insurance, and driver shift wages dominate marginal fuel consumption [3], [4]. Fixed vehicle cost is therefore the main driver, and removing one vehicle saves more, and more regularly, than a marginal distance improvement. Extra compute is therefore worth spending when it removes a vehicle, but the solver that removes it need not be this one. At a planning budget of 5 minutes, the results do not favor a solver on fleet size: none of the three comparators differed detectably from the proposed configuration at 300 s. The choice therefore cannot rest on these results, and the simplest solver to deploy and maintain is a reasonable default; the proposed method adds learned controllers whose benefit this study does not establish. For budgets of seconds, PyVRP-ILS and NeuroLKH-adapted used fewer vehicles than the proposed configuration at 10 s on unadjusted intervals. No 900 s result is reported, so the study gives no guidance for a 15-minute budget.

C. LIMITATIONS AND THREATS TO VALIDITY

The main limitations are these. (i) Benchmark Scope: the evaluation covers 74 instances with four geometry groups per size and few large instances, so it says little about other distributions. (ii) MIP Dependency: Set Partitioning relies on HiGHS throughput within a per-query time limit. (iii) Compute Overhead: online neural updates and Set Partitioning consume part of each wall-clock budget that would otherwise go to search iterations. (iv) Static Scope: the framework targets static VRPTW . (v) GNN Scale Shift: the GNN was trained and validated with one training seed at $N = 100$

and 200 only, so use at $N = 400$ is an extrapolation, and its validation recall is measured against heuristic labels. (vi) LAC Not Exercised: in the cost measurement at 300 s, the LAC classifier never changed an acceptance decision, so these experiments do not test LAC. Because it operates disjunctively (Accept $\iff p_t \geq \tau_{\text{acc}} \vee P_{\text{SA}} > U$), separating predictive lookahead from a higher acceptance rate would also need budgets or instances where it fires. (vii) Effective Compute: machine speed varied between blocks of cells (Section V-A), so results are comparable within a block and do not describe a fixed amount of compute. (viii) One Machine and Fixed Baseline Settings: all runs used one notebook, and the baselines were not tuned for these budgets. (ix) Test-Informed Development: test results motivated the Guided Ejection Search phase, and the development instances share coordinates with test instances (Section V-A). Every incumbent counted in the results is validated in the original problem space by the supervisor before it counts, and cells without a valid incumbent are reported as unavailable.

VII. CONCLUSION

This paper asked whether learned control inside ALNS pays for its compute when the controllers start from random weights in every run. The test case was Tri-Level Hybrid DDQN-ALNS, which adds a macro plateau controller, a micro operator controller and a learned acceptance criterion to ALNS, and every solution counted was validated in the original problem space.

At 300 s, the proposed configuration did not differ detectably on fleet size from ALNS-Base, PyVRP-ILS or NeuroLKH-adapted, and no primary factorial contrast excluded zero, so this study does not establish a benefit of the learned components at that budget. One detectable gain, from the classical Guided Ejection Search phase, was small and did not survive leaving out blocks run at high machine load. A second concerns a learned component: at 60 s, in a separate test on new time-window variants at 200 and 400 customers, Micro-DDQN used fewer vehicles than the rule-based policy in the bare configuration, while the test did not detect a difference in the full context; without the blocks run at high load, neither interval is defined. On descriptive intervals, the proposed configuration used more vehicles than PyVRP-ILS and NeuroLKH-adapted at 10 s, and on the wide-time-window families R2 and RC2 it never used fewer vehicles than NeuroLKH-adapted.

At 300 s, the learned acceptance criterion never changed an acceptance decision and the macro controller was idle on R2 and RC2, so the value of these components remains untested rather than refuted. A test of them needs conditions under which they act, which longer budgets or controllers trained before the run might provide; whether they would help there is open. The benchmark instances, the GNN checkpoint, the solver and baseline adapter code, the evaluation scripts, and compact records of the rerun and of the Micro test (the validated fleet size, distance, timing and load of every cell in the reported stages, together with the analysis outputs and the frozen solver snapshot) are available at https://github.com/Thundercok/vrptw-neural-hybrid-optimizer. The full per-cell outputs, about 23 GB, are available from the authors on request.

ACKNOWLEDGMENT

The authors used Claude (Anthropic) to write analysis and experiment code, to generate the result tables and Figs. 1 and 2, to edit the wording of the manuscript, and to draft parts of the abstract, the introduction and Sections III–VII, which the authors reviewed and revised. The authors verified all output against the experimental records and take full responsibility for the content.

REFERENCES

[1] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
[2] M. M. Solomon, "Algorithms for the vehicle routing and scheduling problems with time window constraints," *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.
[3] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA, USA: SIAM, 2014.
[4] O. Bräysy and M. Gendreau, "Vehicle routing problem with time windows, part I: Route construction and local search algorithms," *Transportation Science*, vol. 39, no. 1, pp. 104–118, 2005.
[5] D. Pisinger and S. Ropke, "A general heuristic for vehicle routing problems," *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.
[6] S. Ropke and D. Pisinger, "An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows," *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.
[7] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2692–2700.
[8] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *International Conference on Learning Representations (ICLR)*, 2019.
[9] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takáč, "Reinforcement learning for solving the vehicle routing problem," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 9861–9871.
[10] J. K. Falkner and L. Schmidt-Thieme, "Learning to solve vehicle routing problems with time windows through joint attention," *arXiv preprint arXiv:2006.09100*, 2020.
[11] B. Lin, B. Ghaddar, and J. Nathwani, "Deep reinforcement learning for the electric vehicle routing problem with time windows," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 11 528–11 538, 2022.
[12] J. Christiaens and G. Vanden Berghe, "Slack induction by string removals for vehicle routing problems," *Transportation Science*, vol. 54, no. 2, pp. 417–433, 2020.
[13] T. Vidal, T. G. Crainic, M. Gendreau, and C. Prins, "A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows," *Computers & Operations Research*, vol. 40, no. 1, pp. 475–489, 2013.
[14] H. Lu, X. Zhang, and S. Yang, "A learning-based iterative method for solving vehicle routing problems," in *International Conference on Learning Representations (ICLR)*, 2020.
[15] A. Hottung and K. Tierney, "Neural large neighborhood search for the capacitated vehicle routing problem," in *Proc. 24th European Conference on Artificial Intelligence (ECAI)*, 2020, pp. 443–450.
[16] S. N. Johnn, V.-A. Darvariu, J. Handl, and J. Kalcsics, "Graph reinforcement learning for operator selection in the ALNS metaheuristic," in *Optimization and Learning (OLA 2023)*, Cham, Switzerland: Springer, 2023, pp. 200–212.
[17] R. Reijnen, Y. Zhang, H. C. Lau, and Z. Bukhsh, "Online control of adaptive large neighborhood search using deep reinforcement learning," in *Proc. Int. Conf. Automated Planning and Scheduling*, vol. 34, no. 1, 2024, pp. 475–483.
[18] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Proc. CP 1998*, ser. LNCS, vol. 1520. Springer, 1998, pp. 417–431.
[19] D. Pisinger and S. Ropke, "Large neighborhood search," in *Handbook of Metaheuristics*. Springer, 2010, pp. 399–419.
[20] Y. Nagata and O. Bräysy, "A powerful route minimization heuristic for the vehicle routing problem with time windows," *Operations Research Letters*, vol. 37, no. 5, pp. 333–338, 2009.

[21] Y. Nagata, O. Bräysy, and W. Dullaert, "A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows," *Computers & Operations Research*, vol. 37, no. 4, pp. 724–737, 2010.

[22] Y. Rochat and E. D. Taillard, "Probabilistic diversification and intensification in local search for vehicle routing," *Journal of Heuristics*, vol. 1, no. 1, pp. 147–167, 1995.

[23] G. B. Alvarenga, G. R. Mateus, and G. de Tomi, "A genetic and set partitioning two-phase approach for the vehicle routing problem with time windows," *Computers & Operations Research*, vol. 34, no. 6, pp. 1561–1584, 2007.

[24] A. Subramanian, E. Uchoa, and L. S. Ochi, "A hybrid algorithm for a class of vehicle routing problems," *Computers & Operations Research*, vol. 40, no. 10, pp. 2519–2531, 2013.

[25] R. Turkeš, K. Sörensen, and L. M. Hvattum, "Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search," *European Journal of Operational Research*, vol. 292, no. 2, pp. 423–442, 2021.

[26] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *arXiv preprint arXiv:1611.09940*, 2016.

[27] E. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 6348–6358.

[28] C. K. Joshi, T. Laurent, and X. Bresson, "An efficient graph convolutional network technique for the travelling salesman problem," *arXiv preprint arXiv:1906.01227*, 2019.

[29] Y. Bengio, A. Lodi, and Y. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.

[30] J. Kallestad, R. Hasibi, A. Hemmati, and K. Sörensen, "A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems," *European Journal of Operational Research*, vol. 309, no. 1, pp. 446–468, 2023.

[31] X. Chen and Y. Tian, "Learning to perform local rewriting for combinatorial optimization," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 6281–6292.

[32] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1-2, pp. 181–211, 1999.

[33] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[34] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artificial Intelligence*, 2016, pp. 2094–2100.

[35] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *International Conference on Learning Representations (ICLR)*, 2016.

[36] L. Li, T. J. Walsh, and M. L. Littman, "Towards a unified theory of state abstraction for MDPs," in *Proc. 9th Int. Symp. Artificial Intelligence and Mathematics (ISAIM)*, 2006, pp. 531–539.

[37] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th ICML*, 1999, pp. 278–287.

[38] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd ICML*, vol. 48, 2016, pp. 1995–2003.

[39] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.

[40] J.-Y. Audibert, R. Munos, and C. Szepesvári, "Exploration–exploitation tradeoff using variance estimates in multi-armed bandits," *Theoretical Computer Science*, vol. 410, no. 19, pp. 1876–1902, 2009.

[41] B. Hajek, "Cooling schedules for optimal annealing," *Mathematics of Operations Research*, vol. 13, no. 2, pp. 311–329, 1988.

[42] M. W. Savelsbergh, "The vehicle routing problem with time windows: Minimizing route duration," *ORSA Journal on Computing*, vol. 4, no. 2, pp. 146–154, 1992.

[43] P. Toth and D. Vigo, "The granular tabu search and its application to the vehicle-routing problem," *INFORMS Journal on Computing*, vol. 15, no. 4, pp. 333–346, 2003.

[44] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[45] H. Gehring and J. Homberger, "A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows," in *Proc. EUROGEN99*, K. Miettinen, M. M. Mäkelä, and J. Toivanen, Eds. University of Jyväskylä, Finland, 1999, pp. 57–64.

[46] N. A. Wouda, L. Lan, and W. Kool, "PyVRP: A high-performance VRP solver package," *INFORMS Journal on Computing*, vol. 36, no. 4, pp. 943–955, 2024.

[47] L. Xin, W. Song, Z. Cao, and J. Zhang, "NeuroLKH: Combining deep learning model with Lin-Kernighan-Helsgaun heuristic for solving the traveling salesman problem," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 7472–7483.

[48] SINTEF Optimization, "SINTEF VRPTW benchmark portal: Best known solutions for Homberger extended instances," https://www.sintef.no/projectweb/top/vrptw/homberger-benchmark/, 2024.



HUYNH NHAT HUY is currently pursuing the B.S. degree in Computer Science with the Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam. His research interests include deep reinforcement learning, combinatorial optimization, metaheuristics, and learning-augmented algorithms.



THI-LINH HO received the B.S. degree from Nong Lam University in 2009, the M.S. degree from Ho Chi Minh City University of Technology in 2012, and the Ph.D. degree in Computer Science from Ton Duc Thang University, Vietnam, in 2024. She is currently a Lecturer with the Faculty of Information Technology, Ton Duc Thang University. Her research interests include machine learning, intelligent information systems, and combinatorial optimization.



NGUYEN NHAT HUY received the B.S. degree in Computer Science from Ton Duc Thang University, Ho Chi Minh City, Vietnam, in 2026. His research interests focus on speech and language processing, generative AI, deep reinforcement learning, and combinatorial optimization.



NGUYEN THI BAO TRAN received the B.S. degree in Software Engineering from Ton Duc Thang University, Ho Chi Minh City, Vietnam, in 2026. Her current research interests include speech and language processing, generative AI, deep reinforcement learning, and combinatorial optimization.